

Université Badji Mokhtar Annaba
Faculté de l'ingéniorat
Département d'informatique
Master 1 Réseaux et sécurité

RMI

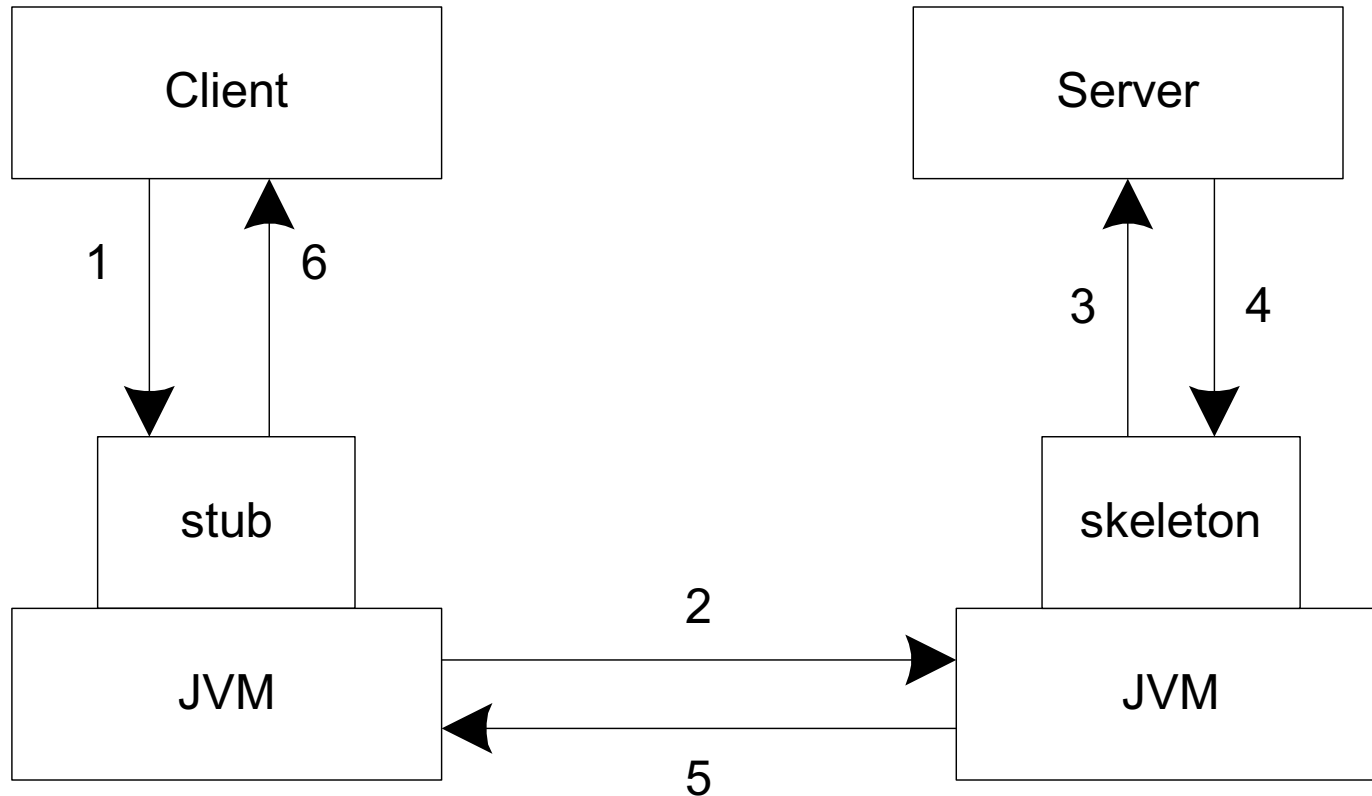
Remote Method Invocation

MAA. S.yahiouche

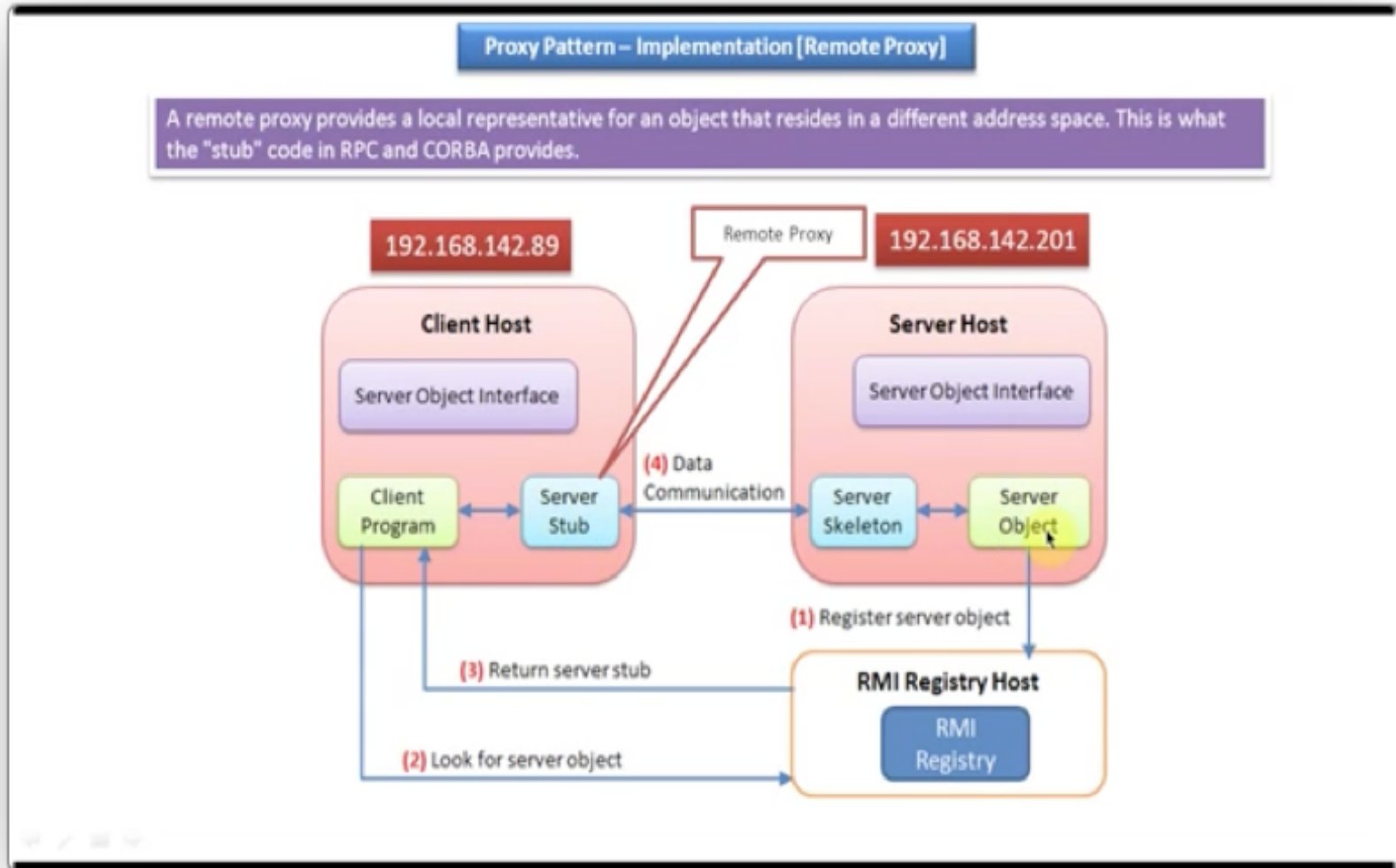
Objectifs

- RMI permet de faire interagir des objets situés dans des espaces d'adressage distincts situés sur des machines distinctes
- RMI repose sur les classes de sérialisation
- Un OD (Objet Distribué) se manipule comme tout autre objet Java

Architecture de l'invocation des méthodes



Architecture de l'invocation des méthodes(version proxy design)



Les processus participants

- Client: invoque une méthode sur un OD
- Serveur: contient l'OD
- Object Registry, **rmiregistry**, (ressemble au service DNS: associe à chaque object un nom), en écoute sur le port 1099 par défaut
- TCP/Ip doit aussi être active

RMI: Deux types de Classes Utiles (Distante/ Sérialisable)

Une classe **distante** (**Remote** class) est une classe dont les instances peuvent être accédées de manière distante

- sur l'ordinateur où elle est définie, les instances de cette classe peuvent être accédées comme n'importe quel autre objet.
- sur d'autres ordinateurs, on peut accéder à l'objet distant à travers des gestionnaires d'objet (**object handles**)

RMI: Deux types de Classes Utiles; (Distante/ Sérialisable)

- Une classe **sérialisable** (Serializable class) est celle dont les instances peuvent être arrangées et ordonnancées en **une séquence linéaire de bits**.
- Les objets sérialisables peuvent être transmis depuis un ordinateur vers un autre

Conditions pour la sérialisabilité

- Si un objet doit être sérialisé, alors:
 - La classe doit être déclarée publique
 - La classe doit implémenter l'interface **Serializable**
 - La classe doit avoir un constructeur sans argument
 - Tous les champs de la classe doivent être sérialisables: qu'ils soient de types primitive ou des objets sérialisables

Interfaces distantes et classe

- Une classe **distante** a deux parties:
 - l'interface (utilisée par le client et le serveur)
 - doit être publique
 - doit étendre l'interface **java.rmi.Remote**
 - chaque méthode dans l'interface doit déclarer qu'elle projette **java.rmi.RemoteException** (d'autres exceptions peuvent aussi être projetées)
 - La classe elle même (utilisée seulement par le serveur):
 - doit implémenter une interface **distante**
 - doit étendre **java.rmi.server.UnicastRemoteObject**
 - peut avoir des méthodes accessibles localement qui ne sont pas dans son interface **distante**

Sécurité

- Il n'est pas prudent pour le client d'utiliser le code de quelqu'un d'autre sur un serveur quelconque
- Votre programme client doit utiliser un gestionnaire de sécurité plus conservatif que celui existant par défaut:
 - `System.setSecurityManager(new RMISecurityManager());`
- La plus part des discussions sur RMI assument que vous devez utiliser ce mécanisme sur le client et sur le serveur
- Hormis le cas dans lequel votre serveur joue aussi le rôle d'un client, ce mécanisme de protection n'est réellement pas nécessaire sur le serveur

La classe serveur

- La classe qui définit l'objet serveur doit étendre **UnicastRemoteObject**:
 - Cela construit une connexion avec exactement **un autre ordinateur**
 - Si vous devez étendre une autre classe, utiliser à la place **exportObject()**
 - Sun ne fournit pas une classe **MulticastRemoteObject**

La classe serveur

- La classe serveur a besoin d'enregistrer son objet serveur:
 - String `url = "rmi://" + host + ":" + port + "/" + objectName;`
 - *le port par défaut est 1099
 - `Naming.rebind(url, object);`
- Chaque méthode disponible de manière distante doit projeter une exception: `RemoteException` (car la connexion peut tomber en panne)
- Chaque méthode disponible de manière distante doit être `synchronisée`

Un programme exemple

- Le client invoque une méthode distante avec **String** comme paramètre
- Le serveur retourne un **String**

Définir l'interface Distante

```
import java.rmi.*;
```

```
public interface HelloInterface extends Remote {  
    public String say() throws RemoteException;  
}
```

Définir la classe Distante

- ```
import java.rmi.*;
import java.rmi.server.*;

public class Hello extends UnicastRemoteObject
 implements HelloInterface {
 private String message; // les Strings sont serializable

 public Hello (String msg) throws RemoteException {
 message = msg;
 }

 public String say() throws RemoteException {
 return message;
 }
}
```

# Le serveur: Enregistrement

- ```
Public class HelloServer {  
    public static void main (String[] argv) {  
If (System.getSecurityManager== null) //optionnel  
    System.setSecurityManager(new RMISecurityManager()); //optionnel  
  
    LocateRegistry.createRegistry(1099);  
  
        try {  
            Naming.rebind("ABC", new Hello("Hello, world!"));  
  
//ou :Naming.rebind("rmi://localhost/HelloServer",new Hello("Hello,  
world!"));  
  
//de manière générale: Naming.rebind(object_location, object);  
  
            System.out.println("Hello Server is ready.");  
        }  
        catch (Exception e) {  
            System.out.println("Hello Server failed: " + e);  
        }  
    }  
}
```


Le Client

- **public class** HelloClient {
 public static void main (String[] args) {
 HelloInterface hello;
 String name = "ABC";
 try {
 hello = (**HelloInterface**)**Naming.lookup**(name);
 System.out.println(hello.say());
 }
 catch (**Exception** e) {
 System.out.println("HelloClient exception: " + e);
 }
 }
}

La compilation

- Sous eclipse, Lancer le **serveur**, puis le **client**.
- La générations des stubs et skeletons est devenue automatique depuis la version 1.5
- Auparavant, on devait utiliser rmic, un compilateur rmi en ligne de commande

Utilisation de rmic

- La classe qui implémente l'objet distant doit être compilée, comme à la normale

Ensuite, elle doit être compilée avec rmic:

-rmic Hello

- Ce qui va générer les fichiers Hello_Stub.class et Hello_Skel.class
- Ce sont ces deux classes qui font réellement la communication
 - La classe “Stub” doit être copiée dans l'espace client
 - La classe “Skel” était utile dans la version 1.1 de SDK , mais elle n'est plus nécessaire

Execution d'une application RMI en ligne de commande

- Dans trois différentes fenêtres Terminales:
 1. Exécuter le programme registry:
 - `Rmiregistry`
 - Dans les nouvelles version, on utilise:
`LocateRegistry.createRegistry(1099)`(voir le code ci-dessus)
 2. Exécuter le programme server:
 - `java HelloServer`
 3. Exécuter le programme client:
 - `java HelloClient`
- Si tous va bien, vous devez avoir le message “Hello, World!”

Résumé

1. Démarrer le serveur registry, **rmiregistry** (dans les anciennes versions)
1. Démarrer le serveur object
 - le serveur object enregistre un objet, par un nom, avec le serveur registry
2. Démarrer le client
 - Le client recherche l'objet dans le serveur registry
3. Le client fait une requête
 1. La requête Va réellement vers la classe stub
 2. Les classes Stub sur le client et le serveur vont se parler
 3. La classe Stub du client retourne le résultat

References

- **Trail: RMI**
by Ann Wollrath and Jim Waldo
 - <http://java.sun.com/docs/books/tutorial/rmi/index.html>
- **Fundamentals of RMI Short Course**
by jGuru
 - <http://developer.java.sun.com/developer/onlineTraining/rmi/RMI.html>
- **Java RMI Tutorial**
by Ken Baclawski
 - http://www.ccs.neu.edu/home/kenb/com3337/rmi_tut.html